

```
# =====  
# DATA CLEANING  
  
# -----  
# 1. IMPORT LIBRARIES  
# -----  
  
import pandas as pd  
import numpy as np  
  
# Display all columns  
pd.set_option("display.max_columns", None)  
|  
# -----  
# 2. LOAD DATASET  
# -----  
  
file_path = "ecommerce_customer_data_custom_ratios.csv"  
  
data = pd.read_csv(file_path)  
  
print("Dataset loaded successfully!")  
print("Original Shape:", data.shape)
```

```
# -----  
# 3. STANDARDIZE COLUMN NAMES  
# -----  
  
# Remove extra spaces, convert to lowercase,  
# and replace spaces with underscores  
  
data.columns = (  
    data.columns  
    .str.strip()  
    .str.lower()  
    .str.replace(" ", "_", regex=False)  
)  
  
print("\nColumns:")  
print(data.columns.tolist())  
  
# -----  
# 4. CREATE DATA COPY  
# -----  
  
# Keep the original data unchanged  
  
df = data.copy()  
  
print("\nWorking dataset shape:", df.shape)
```

```
# -----  
# 5. CONVERT PURCHASE DATE  
# -----  
  
# Convert Purchase Date into datetime format  
  
df["purchase_date"] = pd.to_datetime(  
    df["purchase_date"],  
    errors="coerce"  
)  
  
# Check invalid dates  
  
print(  
    "\nInvalid dates:",  
    df["purchase_date"].isna().sum()  
)  
  
# Display date range  
  
print(  
    "Date Range:",  
    df["purchase_date"].min(),  
    "to",  
    df["purchase_date"].max()  
)
```

```
#
# 6. CONVERT NUMERICAL COLUMNS
# -----

numeric_columns = [
    "product_price",
    "quantity",
    "total_purchase_amount",
    "customer_age",
    "returns",
    "churn"
]

# Convert columns to numeric
# Invalid values become NaN

for column in numeric_columns:
    df[column] = pd.to_numeric(
        df[column],
        errors="coerce"
    )

print("\nNumerical columns converted.")
```

```
# -----  
# 7. CHECK MISSING VALUES  
# -----
```

```
missing_values = pd.DataFrame({  
    "Missing_Count": df.isnull().sum(),  
    "Missing_Percentage": (  
        df.isnull().sum() / len(df) * 100  
    ).round(2)  
})
```

```
print("\nMissing Value Report:")  
print(  
    missing_values.sort_values(  
        "Missing_Percentage",  
        ascending=False  
    )  
)
```

```
#  
# 8. HANDLE MISSING RETURNS  
# -----  
  
# Check return values before treatment  
  
print("\nReturns before treatment:")  
print(df["returns"].value_counts(dropna=False))  
  
# Treat missing return records as no recorded return  
  
df["returns"] = df["returns"].fillna(0)  
  
print(  
    "\nMissing Returns after treatment:",  
    df["returns"].isna().sum()  
)
```

```
# -----  
# 9. HANDLE MISSING CUSTOMER AGE  
# -----  
  
# Replace missing ages with the median age  
  
if df["customer_age"].isna().sum() > 0:  
  
    median_age = df["customer_age"].median()  
  
    df["customer_age"] = df["customer_age"].fillna(  
        median_age  
    )  
  
print(  
    "\nMissing Customer Age:",  
    df["customer_age"].isna().sum()  
)
```

```
# -----  
# 10. HANDLE MISSING CHURN  
# -----  
  
# Check churn values  
  
print("\nChurn values:")  
print(df["churn"].value_counts(dropna=False))  
  
# If churn contains missing values,  
# remove those records because churn is an important  
# target variable and should not be guessed  
  
df = df.dropna(subset=["churn"])
```

```
# -----  
# 11. REMOVE RECORDS WITH INVALID DATES  
# -----
```

```
# Purchase date is essential for recency calculation
```

```
df = df.dropna(  
    subset=["purchase_date"]  
)  
  
print(  
    "\nShape after date cleaning:",  
    df.shape  
)
```

```
# -----  
# 12. CHECK DUPLICATE ROWS  
# -----  
  
duplicate_count = df.duplicated().sum()  
  
print(  
    "\nDuplicate rows:",  
    duplicate_count  
)  
  
# Remove exact duplicate rows  
  
df = df.drop_duplicates()  
  
print(  
    "Shape after duplicate removal:",  
    df.shape  
)  
  
#
```

```
# -----  
# 13. CHECK INVALID QUANTITY  
# -----  
  
print(  
    "\nInvalid Quantity:",  
    (df["quantity"] <= 0).sum()  
)  
  
# Remove transactions with zero/negative quantity  
  
df = df[df["quantity"] > 0]  
  
# -----  
# 14. CHECK INVALID PRODUCT PRICE  
# -----  
  
print(  
    "\nInvalid Product Price:",  
    (df["product_price"] <= 0).sum()  
)  
  
# Remove transactions with zero/negative price  
  
df = df[df["product_price"] > 0]
```

```
# -----  
# 15. CHECK INVALID PURCHASE AMOUNT  
# -----  
  
print(  
    "\nInvalid Purchase Amount:",  
    (df["total_purchase_amount"] <= 0).sum()  
)  
  
# Remove invalid purchase amounts  
  
df = df[  
    df["total_purchase_amount"] > 0  
]
```

```
#  
# 16. CHECK CUSTOMER AGE RANGE  
# -----
```

```
invalid_age = (  
    (df["customer_age"] < 18) |  
    (df["customer_age"] > 100)  
)
```

```
print(  
    "\nInvalid age records:",  
    invalid_age.sum()  
)
```

```
# Remove impossible age values
```

```
df = df[  
    ~invalid_age  
]
```

```
# -----  
# 17. CHECK AGE COLUMN  
# -----
```

```
# The dataset contains both customer_age and age.  
# Check whether they contain the same information.
```

```
if "age" in df.columns:  
  
    age_difference = (  
        df["customer_age"] != df["age"]  
    ).sum()  
  
    print(  
        "\nAge mismatches:",  
        age_difference  
    )  
  
    # Remove duplicate age column  
  
    df = df.drop(  
        columns=["age"]  
    )
```

```
# -----  
# 18. CLEAN CATEGORICAL COLUMNS  
# -----
```

```
categorical_columns = [  
    "product_category",  
    "payment_method",  
    "gender"  
]  
  
for column in categorical_columns:  
  
    # Remove extra spaces  
  
    df[column] = (  
        df[column]  
        .astype(str)  
        .str.strip()  
    )
```

```
# -----  
# 19. CHECK CUSTOMER IDs  
# -----  
  
print(  
    "\nMissing Customer IDs:",  
    df["customer_id"].isna().sum()  
)  
  
# Remove transactions without customer ID  
  
df = df.dropna(  
    subset=["customer_id"]  
)  
  
# -----  
# 20. CHECK DATASET AFTER CLEANING  
# -----  
  
print("\nFinal Cleaned Transaction Shape:")  
print(df.shape)  
  
print("\nRemaining Missing Values:")  
print(df.isnull().sum())
```

```
# 21. CREATE DATE FEATURES
```

```
# -----
```

```
df["year"] = df["purchase_date"].dt.year
```

```
df["month"] = df["purchase_date"].dt.month
```

```
df["month_name"] = (  
    df["purchase_date"].dt.month_name()  
)
```

```
df["quarter"] = (  
    df["purchase_date"].dt.quarter  
)
```

```
df["day_of_week"] = (  
    df["purchase_date"].dt.day_name()  
)
```

```
#
# 22. CREATE CALCULATED PURCHASE AMOUNT
# -----

# Calculate expected transaction amount

df["calculated_amount"] = (
    df["product_price"] *
    df["quantity"]
)

# Calculate difference from recorded amount

df["amount_difference"] = (
    df["total_purchase_amount"] -
    df["calculated_amount"]
)

print("\nPurchase Amount Validation:")
print(
    df["amount_difference"].describe()
)
```

```
# -----  
# 23. REMOVE TEMPORARY VALIDATION COLUMNS  
# -----
```

```
df = df.drop(  
    columns=[  
        "calculated_amount",  
        "amount_difference"  
    ]  
)
```

```
#  
# 24. CREATE CUSTOMER-LEVEL DATASET
```

```
# -----  
  
customer_df = (  
    df.groupby(  
        ["customer_id", "customer_name"]  
    )  
    .agg(  
        total_spend=(  
            "total_purchase_amount",  
            "sum"  
        ),  
  
        purchase_frequency=(  
            "customer_id",  
            "count"  
        ),  
  
        total_quantity=(  
            "quantity",  
            "sum"  
        ),  
  
        average_order_value=(  
            "total_purchase_amount",  
            "mean"  
        ),  
    )  
)
```

```
average_product_price=(  
    "product_price",  
    "mean"  
)
```

```
first_purchase_date=(  
    "purchase_date",  
    "min"  
)
```

```
last_purchase_date=(  
    "purchase_date",  
    "max"  
)
```

```
total_returns=(  
    "returns",  
    "sum"  
)
```

```
churn=(  
    "churn",  
    "max"  
)
```

```
)  
.reset_index()
```

```
)
```

```
# -----  
# 25. CREATE RECENCY  
# -----  
  
# Use the latest transaction date as the analysis date  
  
analysis_date = df["purchase_date"].max()  
  
print(  
    "\nAnalysis Date:",  
    analysis_date  
)  
  
# Calculate days since last purchase  
  
customer_df["recency"] = (  
    analysis_date -  
    customer_df["last_purchase_date"]  
) .dt.days
```

```
# -----  
# 26. CREATE CUSTOMER TENURE  
# -----  
  
# Number of days between first and last purchase  
  
customer_df["customer_tenure_days"] = (  
    customer_df["last_purchase_date"] -  
    customer_df["first_purchase_date"]  
) .dt.days  
  
# Convert tenure into months  
  
customer_df["customer_tenure_months"] = (  
    customer_df["customer_tenure_days"] / 30  
) .round(1)  
  
# -----
```

```
# -----  
# 27. CREATE RETURN RATE  
# -----
```

```
customer_df["return_rate"] = (  
    customer_df["total_returns"] /  
    customer_df["purchase_frequency"]  
)
```

```
# Convert to percentage
```

```
customer_df["return_rate_pct"] = (  
    customer_df["return_rate"] * 100  
)  
.round(2)
```

```
|
```

```
# -----  
# 28. CUSTOMER AGE AND GENDER
```

```
# -----  
  
customer_info = (  
    df.groupby("customer_id")  
    .agg(  
        customer_age=(  
            "customer_age",  
            "first"  
        ),  
  
        gender=(  
            "gender",  
            "first"  
        )  
    )  
    .reset_index()  
)  
  
customer_df = customer_df.merge(  
    customer_info,  
    on="customer_id",  
    how="left"  
)
```

```
# 29. PREFERRED PRODUCT CATEGORY
```

```
# -----
```

```
# Find the most frequently purchased category
```

```
# for each customer
```

```
category_preference = (  
    df.groupby("customer_id")["product_category"]  
    .agg(  
        lambda x:  
            x.mode().iloc[0]  
            if not x.mode().empty  
            else "Unknown"  
    )  
    .reset_index()  
)
```

```
category_preference.columns = [  
    "customer_id",  
    "preferred_category"  
]
```

```
customer_df = customer_df.merge(  
    category_preference,  
    on="customer_id",  
    how="left"  
)
```

```
# 30. PREFERRED PAYMENT METHOD
# -----

# Find the most frequently used payment method

payment_preference = (
    df.groupby("customer_id")["payment_method"]
    .agg(
        lambda x:
            x.mode().iloc[0]
            if not x.mode().empty
            else "Unknown"
    )
    .reset_index()
)

payment_preference.columns = [
    "customer_id",
    "preferred_payment_method"
]

customer_df = customer_df.merge(
    payment_preference,
    on="customer_id",
    how="left"
)
```

```
# 31. VERIFY CUSTOMER-LEVEL DATA
```

```
# -----
```

```
print("\nCustomer-Level Dataset Shape:")
```

```
print(customer_df.shape)
```

```
print("\nCustomer-Level Columns:")
```

```
print(customer_df.columns.tolist())
```

```
print("\nFirst 5 Customers:")
```

```
print(customer_df.head())
```

```
# -----
```

```
# 32. CHECK DUPLICATE CUSTOMERS
```

```
# -----
```

```
duplicate_customers = (  
    customer_df["customer_id"]  
    .duplicated()  
    .sum()  
)
```

```
print(  
    "\nDuplicate Customers:",  
    duplicate_customers  
)
```

```
# 33. CUSTOMER STATISTICAL SUMMARY
```

```
# -----
```

```
customer_numeric_columns = [  
    "total_spend",  
    "purchase_frequency",  
    "total_quantity",  
    "average_order_value",  
    "average_product_price",  
    "recency",  
    "customer_tenure_days",  
    "total_returns",  
    "return_rate_pct"  
]
```

```
print("\nCustomer Statistics:")
```

```
print(  
    customer_df[  
        customer_numeric_columns  
    ].describe()  
)
```

```
#
```

```
# 34. OUTLIER DETECTION
```

```
# -----
```

```
# Detect potential outliers using IQR.
```

```
# Valid high-value customers will NOT be removed.
```

```
for column in [  
    "total_spend",  
    "purchase_frequency",  
    "total_quantity",  
    "average_order_value",  
    "recency"  
]:  
  
    Q1 = customer_df[column].quantile(0.25)  
  
    Q3 = customer_df[column].quantile(0.75)  
  
    IQR = Q3 - Q1  
  
    lower_limit = Q1 - 1.5 * IQR  
  
    upper_limit = Q3 + 1.5 * IQR  
  
    outliers = (  
        (customer_df[column] < lower_limit) |  
        (customer_df[column] > upper_limit)  
    ).sum()
```

```
).sum()
```

```
print(
```

```
    f"{column}: {outliers} potential outliers"
```

```
)
```

```
# 35. CREATE RFM BASE DATASET
```

```
# -----
```

```
rfm = customer_df[  
    [  
        "customer_id",  
        "customer_name",  
        "recency",  
        "purchase_frequency",  
        "total_spend",  
        "churn"  
    ]  
].copy()
```

```
# Rename columns according to RFM terminology
```

```
rfm = rfm.rename(  
    columns={  
        "purchase_frequency": "frequency",  
        "total_spend": "monetary"  
    }  
)
```

```
# 36. CHECK RFM DATA
```

```
# -----  
  
print("\nRFM Dataset Shape:")
```

```
print(rfm.shape)
```

```
  
print("\nRFM Dataset:")
```

```
print(rfm.head())
```

```
  
print("\nRFM Missing Values:")
```

```
print(rfm.isnull().sum())
```

```
# 37. SAVE CLEANED TRANSACTION DATA
```

```
# -----
```

```
df.to_csv(  
    "ecommerce_cleaned_transactions.csv",  
    index=False  
)  
  
print(  
    "\nCleaned transaction dataset saved!"  
)
```

```
# -----
```

```
# 38. SAVE CUSTOMER-LEVEL FEATURES
```

```
# -----
```

```
customer_df.to_csv(  
    "customer_level_features.csv",  
    index=False  
)  
  
print(  
    "Customer-level feature dataset saved!"  
)
```

```
# -----  
# 39. SAVE RFM BASE DATASET  
# -----
```

```
rfm.to_csv(  
    "customer_rfm_base.csv",  
    index=False  
)  
  
print(  
    "RFM base dataset saved!"  
)
```

```
# =====  
# FINAL SUMMARY  
# =====  
  
print("\n" + "=" * 60)  
print("PHASE 2 COMPLETED")  
print("=" * 60)  
  
print(  
    "Cleaned Transactions:",  
    len(df)  
)  
  
print(  
    "Unique Customers:",  
    customer_df["customer_id"].nunique()  
)  
  
print(  
    "RFM Customers:",  
    len(rfm)  
)  
  
print(  
    "Analysis Date:",  
    analysis_date  
)
```

```
print("\nOutput Files:")
print("1. ecommerce_cleaned_transactions.csv")
print("2. customer_level_features.csv")
print("3. customer_rfm_base.csv")
```

Dataset loaded successfully!

Original Shape: (250000, 13)

Columns:

```
['customer_id', 'purchase_date', 'product_category', 'product_price', 'quantity', 'total_purchase_amount', 'payment_method', 'customer_age', 'returns', 'customer_name', 'age', 'gender', 'churn']
```

Working dataset shape: (250000, 13)

Invalid dates: 0

Date Range: 2020-01-01 00:15:00 to 2023-09-15 12:24:08

Numerical columns converted.

Missing Value Report:

	Missing_Count	Missing_Percentage
returns	47596	19.04
customer_id	0	0.00
purchase_date	0	0.00
product_category	0	0.00
product_price	0	0.00
quantity	0	0.00
total_purchase_amount	0	0.00
payment_method	0	0.00
customer_age	0	0.00
customer_name	0	0.00
age	0	0.00
gender	0	0.00
churn	0	0.00

Returns before treatment:

returns

0.0 101635

1.0 100769

NaN 47596

Name: count, dtype: int64

Missing Returns after treatment: 0

Missing Customer Age: 0

Churn values:

churn

0 200126

1 49874

Name: count, dtype: int64

Shape after date cleaning: (250000, 13)

Duplicate rows: 0

Shape after duplicate removal: (250000, 13)

Invalid Quantity: 0

Invalid Product Price: 0

Invalid Purchase Amount: 0

Invalid age records: 0

Age mismatches: 0

Missing Customer IDs: 0

Final Cleaned Transaction Shape:

(250000, 12)

Remaining Missing Values:

customer_id	0
purchase_date	0
product_category	0
product_price	0
quantity	0
total_purchase_amount	0
payment_method	0
customer_age	0
returns	0
customer_name	0
gender	0
churn	0

dtype: int64

Purchase Amount Validation:

count	250000.000000
mean	1961.734976
std	1560.451368
min	-2330.000000
25%	715.000000
50%	1962.000000
75%	3219.000000
max	5314.000000

Name: amount_difference, dtype: float64

Customer-Level Dataset Shape:
(49673, 20)

Customer-Level Columns:
['customer_id', 'customer_name', 'total_spend', 'purchase_frequency', 'total_quantity', 'average_order_value', 'average_product_price', 'first_purchase_date', 'last_purchase_date', 'total_returns', 'churn', 'recency', 'customer_tenure_days', 'customer_tenure_months', 'return_rate', 'return_rate_pct', 'customer_age', 'gender', 'preferred_category', 'preferred_payment_method']

First 5 Customers:

	customer_id	customer_name	total_spend	purchase_frequency	\
0	1	Nicole Johnson	3491	1	
1	2	Marie Wright	7988	3	
2	3	Julie Wolfe	22587	8	
3	4	Tracey Smith	8715	4	
4	5	Nancy Jones	12524	8	

	total_quantity	average_order_value	average_product_price	\
0	5	3491.000000	169.000	
1	8	2662.666667	140.000	
2	23	2823.375000	225.375	
3	9	2178.750000	134.250	
4	33	1565.500000	229.000	

	first_purchase_date	last_purchase_date	total_returns	churn	recency	\
0	2023-07-19 19:09:20	2023-07-19 19:09:20	1.0	0	57	
1	2021-03-05 14:30:42	2022-11-21 06:48:05	1.0	0	298	
2	2020-09-27 22:26:24	2023-06-18 12:31:42	3.0	0	88	
3	2020-10-09 03:25:04	2023-05-12 10:50:24	1.0	1	126	
4	2020-01-28 14:28:33	2023-03-28 13:30:55	4.0	0	170	

	customer_tenure_days	customer_tenure_months	return_rate	return_rate_pct	\
0	0	0.0	1.000000	100.00	
1	625	20.8	0.333333	33.33	
2	993	33.1	0.375000	37.50	
3	945	31.5	0.250000	25.00	
4	1154	38.5	0.500000	50.00	

	customer_age	gender	preferred_category	preferred_payment_method
0	70	Male	Books	PayPal
1	27	Female	Books	PayPal
2	23	Female	Clothing	Credit Card
3	66	Male	Clothing	Credit Card
4	26	Female	Clothing	Credit Card

Duplicate Customers: 0

Customer Statistics:

	total_spend	purchase_frequency	total_quantity	average_order_value	\
count	49673.000000	49673.000000	49673.000000	49673.000000	
mean	13716.559962	5.032915	15.093189	2724.642085	
std	6840.208557	2.206427	7.306184	737.013884	
min	110.000000	1.000000	1.000000	110.000000	
25%	8719.000000	3.000000	10.000000	2255.714286	
50%	13026.000000	5.000000	14.000000	2728.600000	
75%	17966.000000	6.000000	20.000000	3191.200000	
max	55339.000000	17.000000	55.000000	5288.000000	

	average_product_price	recency	customer_tenure_days	\
count	49673.000000	49673.000000	49673.000000	
mean	254.409306	260.424979	830.876210	
std	72.043251	246.024001	337.976142	
min	10.000000	0.000000	0.000000	
25%	208.625000	77.000000	631.000000	
50%	254.600000	184.000000	904.000000	
75%	300.500000	368.000000	1095.000000	
max	500.000000	1352.000000	1351.000000	

	total_returns	return_rate_pct
count	49673.000000	49673.000000
mean	2.028647	40.156954
std	1.421131	24.895908
min	0.000000	0.000000
25%	1.000000	25.000000
50%	2.000000	40.000000
75%	3.000000	55.560000
max	10.000000	100.000000

total_spend: 569 potential outliers
 purchase_frequency: 677 potential outliers
 total_quantity: 411 potential outliers
 average_order_value: 944 potential outliers
 recency: 2187 potential outliers

RFM Dataset Shape:
 (49673, 6)

RFM Dataset:

	customer_id	customer_name	recency	frequency	monetary	churn
0	1	Nicole Johnson	57	1	3491	0
1	2	Marie Wright	298	3	7988	0
2	3	Julie Wolfe	88	8	22587	0
3	4	Tracey Smith	126	4	8715	1
4	5	Nancy Jones	170	8	12524	0

RFM Missing Values:

customer_id	0
customer_name	0
recency	0
frequency	0
monetary	0
churn	0

dtype: int64

Cleaned transaction dataset saved!
Customer-level feature dataset saved!
RFM base dataset saved!

=====
PHASE 2 COMPLETED

=====
Cleaned Transactions: 250000
Unique Customers: 49673
RFM Customers: 49673
Analysis Date: 2023-09-15 12:24:08

Output Files:

1. ecommerce_cleaned_transactions.csv
2. customer_level_features.csv
3. customer_rfm_base.csv